005-Slice

Difficulty: Hard Expect Time: 20-40 min Author: Yan Hau, Chen

In this problem, you are asked to implement a generic container class **`Slice<T>`** that supports element access via a custom random-access iterator and provides a set of functional utilities similar to those found in modern programming languages.

- Iterator Implementation

  Complete the implementation of the template class **`Iterator<T>`** in iterator.h. This iterator should conform to the **`RandomAccessIterator`** category and support the following operator:

  - Dereferencing and pointer-like access(**`*`**, **`->`**)
  - Pre/post increment and decrement (**`++`**, **`--`**)
  - Offset arithmetic (**`+, -, +=, -=`**) and subscript access (**`[]`**)
  - Comparison operators (**`==, !=, <, <=, >, >=`**)
  - **Compatibility with standard algorithms** such as **`std::find`**, **`std::find_if`** etc.

  Once the iterator is complete, implement the following member functions in **`Slice<T>`**:

  - **`begin()`** – Returns an **iterator** to the first element.
  - **`end()`** – Returns an **iterator** one-past-the-last element.
  - **`operator[](size_t index)`** – Provides random access to elements by index.

- Functional Utilities

  Implement the following higher-order member functions in **`Slice<T>`**, They accept a Functor as a parameter:

  - **`map(fn: T → U) → Slice<U>`**

    Applies a transformation function to each element and returns a new **`Slice<U>`** containing the results.

  - **`filter(fn: Predicate) → Slice<T>`**

    Applies a predicate function to each element and returns a new **`Slice<T>`** containing only the elements for which the **`Predicate`** returns true.

  - **`every(fn: Predicate) → bool`**

    Returns **`true`** if all elements satisfy the **`Predicate`**; otherwise, returns **`false.`**

- Type Alias
  - **`template<typename K> Storage`**
    Please redefine as the container type used to store internal data.
  - **`Predicate`**
    Please infer the appropriate type for **`Predicate`**. based on its use in functions like **`every()`** and **`filter()`**.

  Both types will be publicly exposed via **using** declarations.

`slice.h` already contains the declarations of essential members for the **`Slice`** class. You are expected to complete their implementations based on the provided interface.

Similarly, **`iterator.h`** includes a partial implementation of a custom iterator. Please complete the missing parts so that the iterator satisfies the requirements of a random access iterator.

If you find that the current design of the **`Slice`** class is insufficient to meet the problem requirements, **ALLOWED** to add more members, methods, constructors, or supporting structures as needed.

Please ensure that your implementation is **compatible** with **`range-based for loops`** and standard algorithms from **`<algorithm>`**. You may refer to the examples provided in **`case.h`** for guidance.

Input

1. Complete the required class implementation. The following files already contain some implementations
   a) **`iterator.h`**
   b) **`slice.h`**
   You can submit a header-only implementation, or split the implementation into **`*.cpp`** files.

2. The Online Judge system will replace the following files:
   a) **`entrypoint.cpp`**
   b) **`case.h`**
   c) **`test.h`**

3. The test cases for student available in **`case.h`**, but not all test data is included in that file. The Online Judge system will also include hidden test inputs.

4. All template parameters **`T`** used in this problem are required to be **`default-initializable`**. That is, the expression **`T{}`** must be valid and produce a default value of type **`T`**.

5. The functors passed to `` `map` ``, `` `filter` ``, and `` `every` `` must have clearly defined parameter and return types, so that their usage can be type-checked safely.

Output

Online Judge will run the test:

1. **DON' T** output any information to the terminal.
2. When any test failed, Online Judge will try to list error messages.